ACADEMIC YEAR 2025 – 2026

10212CS211: Artificial Intelligence Techniques

Lab Tasks Reference & Code Memorization Guide

Course Code:	10212CS211
Course Title:	Artificial Intelligence Techniques (AIT)
Department:	Computer Science and Engineering
Programming Language:	Python 3.x
Coverage:	Tasks 1 to 10 (Full Manual)
Content:	Question / Aim, Source Code, Memory Hacks & Logic Breakdown

Graph Search (BFS & DFS) | Heuristic Hill Climbing | A* Search | Minimax Game Tree | Ant Colony Optimization (ACO) | Constraint Satisfaction | Goal Stack Planning | N-Queens Backtracking | Conversational AI

TASK 1 (PART A & PART B)

Graph Search Algorithms: BFS & DFS

Aim (Part 1a - Stock Market BFS): To determine the set of companies impacted within a specified depth (levels of ripple effect) when a given company experiences a significant event, using Breadth-First Search (BFS).
Aim (Part 1b - Weather Forecasting DFS): To traverse a network of weather stations using Depth-First Search (DFS) and display the weather status of each station starting from a designated station.

► Python Program

```
from collections import deque

# === Part 1a: BFS for Stock Market Influence ===
graph = {
    "Company A": ["Company B", "Company C"],
    "Company B": ["Company D"],
    "Company C": ["Company D"],
    "Company D": ["Company E", "Company F"],
    "Company E": [], "Company F": [], "Company G": []
}

def bfs_influence(graph, start_company, max_depth=2):
    queue = deque([[start_company, 0]])   # [node, depth]
    visited = set([start_company])
    impacted = []

    while queue:
        current, depth = queue.popleft()
        if depth == max_depth:
            continue
        for neighbor in graph.get(current, []):
            if neighbor not in visited:
                visited.add(neighbor)
                impacted.append(neighbor)
                queue.append([neighbor, depth + 1])
    return impacted

# === Part 1b: DFS for Weather Stations ===
weather_graph = {
    'station A': ['station B', 'station C'],
    'station B': ['station D'],
    'station C': ['station F'],
    'station D': ['station E'],
    'station E': [], 'station F': [], 'station P': []
}
weather_data = {
    'station A': 'sunny', 'station B': 'cloudy',
    'station C': 'rainy', 'station D': 'stormy',
    'station E': 'windy', 'station F': 'snowy'
}

def dfs(graph, node, visited):
    if node not in visited:
        print(f"{node} reports: {weather_data[node]}")
```

```
        visited.add(node)
        for neighbor in graph.get(node, []):
            dfs(graph, neighbor, visited)
```

💡 Easy Memory Hook & How to Remember the Code:

- **BFS = Queue (FIFO) + Deque:** Remember "*BFS spreads like water ripples in all directions*". Store `[company, depth]` in a queue. Pop left, if `depth == max_depth` stop expanding that branch, otherwise enqueue neighbors with `depth + 1`.
- **DFS = Recursion (Stack / LIFO):** Remember "*DFS goes deep down a rabbit hole first*". Check if not visited → print weather → mark visited → recursively call `dfs(graph, neighbor, visited)`.
- **Data structures:** Always use a `set()` for `visited` to prevent infinite cycles in graphs!

❓ Quick Viva Questions:

Q: Why use BFS for stock market impact?
A: Stock influence cascades level-by-level (direct partners first at depth 1, secondary partners at depth 2). BFS explores by level.
Q: What is the time complexity of BFS/DFS?
A: $O(V + E)$ where V is vertices/companies and E is edges/connections.

TASK 2

Vehicle Routing using Hill Climbing Algorithm

Aim: To find an optimized (shortest) route for visiting multiple customers using the Hill Climbing heuristic search algorithm, which iteratively explores neighboring routes and adopts improvements until reaching a local optimum.

► Python Program

```
import random

def generate_random_route(num_customers):
    customers = list(range(1, num_customers + 1))
    random.shuffle(customers)
    return [0] + customers + [0]  # Start at depot (0) and return to 0

def generate_neighbors(route):
    neighbors = []
    n = len(route)
    for i in range(1, n - 1):
        for j in range(i + 1, n - 1):
            neighbor = route.copy()
            neighbor[i], neighbor[j] = neighbor[j], neighbor[i]  # Swap 2 customers
            neighbors.append(neighbor)
    return neighbors

def calculate_total_distance(route, dist_matrix):
    return sum(dist_matrix[route[i]][route[i+1]] for i in range(len(route)-1))

def hill_climbing(dist_matrix, initial_route):
    current_route = initial_route
    current_distance = calculate_total_distance(current_route, dist_matrix)
    improving = True

    while improving:
        improving = False
        neighbors = generate_neighbors(current_route)
        for neighbor in neighbors:
            neighbor_distance = calculate_total_distance(neighbor, dist_matrix)
            if neighbor_distance < current_distance:
                current_route = neighbor
                current_distance = neighbor_distance
                improving = True
                break  # First-choice / greedy acceptance
    return current_route, current_distance
```

💡 **Easy Memory Hook & How to Remember the Code:**

- **4 Key Functions:**
    1. `generate_random_route()` → Shuffle numbers 1 to N, sandwich between `[0] + ... + [0]`.
    2. `calculate_total_distance()` → Sum edge weights from node i to node $i+1$.

3. `generate_neighbors()` → Nested loops (i, j) swapping two positions inside the route.
4. `hill_climbing()` → `while improving:` loop. If any neighbor is shorter, accept it and break immediately.

- **Analogy:** Walking downhill in dense fog. Take a step; if the ground goes lower, move there. Stop when every step in all directions goes up (local minimum).

❓ Quick Viva Questions:

Q: What is the main drawback of standard Hill Climbing?
A: It can get stuck in a **Local Optimum** (or plateau/ridge) instead of reaching the global optimum.
Q: How can local optima be escaped?
A: Using Random-Restart Hill Climbing, Simulated Annealing, or Genetic Algorithms.

TASK 3

A* Search Algorithm (Map Navigation)

Aim: To find the optimal (shortest-cost) path between a start node and goal node in a weighted graph using the A* search algorithm, combining the exact accumulated path cost $g(n)$ and heuristic estimate $h(n)$.

► Python Program

```
import heapq

graph = {
    'A': {'B': 6, 'F': 3}, 'B': {'C': 1, 'G': 6, 'F': 3},
    'C': {'D': 1, 'E': 5}, 'D': {'E': 8},
    'E': {'I': 5, 'J': 5}, 'F': {'G': 1, 'H': 7},
    'G': {'H': 3},         'H': {'I': 2},
    'I': {'H': 2, 'J': 3}, 'J': {}
}

position = {'A': 10, 'B': 8, 'C': 5, 'D': 7, 'E': 3, 'F': 6, 'G': 5, 'H': 3, 'I': 1, 'J': 0}

def heuristic(a, b):
    return abs(position[a] - position[b])

def a_star(graph, start, goal):
    open_set = []
    heapq.heappush(open_set, (heuristic(start, goal), 0, start)) # (f, g, node)
    came_from = {}
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    while open_set:
        f, g, current = heapq.heappop(open_set)
        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1], g_score[goal]

        for neighbor, cost in graph[current].items():
            tentative_g = g_score[current] + cost
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score = tentative_g + heuristic(neighbor, goal)
                heapq.heappush(open_set, (f_score, tentative_g, neighbor))
    return None, float('inf')
```

💡 Easy Memory Hook & How to Remember the Code:

- **The Golden Equation:** f(n) = g(n) + h(n)

    - `g(n)` = Exact distance travelled from start so far.
    - `h(n)` = Heuristic estimated straight-line distance to goal.
    - `f(n)` = Total estimated cost → Always pop the smallest f from the Priority Queue (`heapq`).
- **Track Parents with `came_from`:** To reconstruct the path once you reach the goal, backtrack through `came_from` dictionary and reverse it: `path[::-1]`.

❓ Quick Viva Questions:

Q: When is A* guaranteed to find the optimal path?
A: When the heuristic is **admissible** (it never overestimates the true cost to reach the goal).

TASK 4

Tic-Tac-Toe using Mini-Max Algorithm

Aim: To develop an unbeatable Tic-Tac-Toe game where the computer (AI, 'X') plays optimally using the recursive Minimax algorithm, ensuring it either wins or ties against a human player ('O').

► Python Program

```
import math

def check_winner(board):
    wins = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]]
    for line in wins:
        if board[line[0]] == board[line[1]] == board[line[2]] != ' ':
            return board[line[0]]
    if ' ' not in board: return 'Tie'
    return None

def minimax(board, is_maximizing):
    winner = check_winner(board)
    if winner == 'X': return 1       # AI wins
    if winner == 'O': return -1      # Human wins
    if winner == 'Tie': return 0

    if is_maximizing:  # AI's turn (maximize score)
        best_score = -math.inf
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(board, False)
                board[i] = ' '        # Backtrack
                best_score = max(score, best_score)
        return best_score
    else:              # Human's turn (minimize score)
        best_score = math.inf
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(board, True)
                board[i] = ' '        # Backtrack
                best_score = min(score, best_score)
        return best_score

def best_move(board):
    best_score, move = -math.inf, None
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(board, False)
            board[i] = ' '
            if score > best_score:
                best_score, move = score, i
    return move
```

💡 Easy Memory Hook & How to Remember the Code:

- **Terminal Values:** AI win = `+1`, Human win = `-1`, Draw = `0`.
- **Two Roles:**
  - `is_maximizing == True` (AI): Wants highest score → `max(score, best_score)`.
  - `is_maximizing == False` (Player): Wants lowest score → `min(score, best_score)`.
- **Always Unmake Moves:** Set `board[i] = 'X'`, call recursion, then immediately reset `board[i] = ' '` (backtracking).

❓ Quick Viva Questions:

Q: How can Minimax be optimized?

A: By implementing **Alpha-Beta Pruning** to avoid exploring branches that cannot influence the final decision.

TASK 5

Ant Colony Optimization (ACO) for Route Optimization

Aim: To find the shortest delivery route that visits all given cities once and returns to the starting depot using Ant Colony Optimization (ACO) inspired by the foraging behavior of real ants.

► Python Program

```
import numpy as np
import random

cities = ["Depot", "Mumbai", "Delhi", "Chennai", "Kolkata",
          "Bangalore", "Hyderabad", "Pune", "Ahmedabad", "Jaipur", "Lucknow"]
# dist_matrix is an 11x11 numpy array of distances

n_locations, n_ants, n_iterations = len(cities), 20, 100
alpha, beta, rho, Q = 1.0, 5.0, 0.5, 100
pheromone = np.ones((n_locations, n_locations))

def ant_tour():
    tour = [0]
    unvisited = list(range(1, n_locations))
    while unvisited:
        i = tour[-1]
        probs = []
        for j in unvisited:
            tau = pheromone[i, j] ** alpha            # Pheromone factor
            eta = (1.0 / dist_matrix[i, j]) ** beta   # Heuristic visibility (1/distance)
            probs.append(tau * eta)
        probs = np.array(probs) / np.sum(probs)      # Normalize probabilities
        next_city = random.choices(unvisited, weights=probs, k=1)[0]
        tour.append(next_city)
        unvisited.remove(next_city)
    tour.append(0)  # Return to depot
    return tour

# ACO Simulation Loop
best_route, best_length = None, float("inf")
for _ in range(n_iterations):
    routes, lengths = [], []
    for _ in range(n_ants):
        t = ant_tour()
        L = sum(dist_matrix[t[k], t[k+1]] for k in range(len(t)-1))
        routes.append(t); lengths.append(L)
        if L < best_length: best_length, best_route = L, t

    pheromone *= (1 - rho)  # Evaporation
    for r, L in zip(routes, lengths):
        for k in range(len(r)-1):
            pheromone[r[k], r[k+1]] += Q / L  # Deposit pheromone
```

💡 **Easy Memory Hook & How to Remember the Code:**

- **The Core Probability Rule:** $P_{ij} \propto (au_{ij})^□lpha imes (\eta_{ij})^□eta$
  - $ au$ (tau) = Pheromone (history of where other ants succeeded).
  - η (eta) = Visibility ($1 / ext{distance}$) → Closer cities have higher attraction.
- **Two Essential Updates:**
  1. `pheromone *= (1 - rho)` → Evaporation (old trails fade away).
  2. `pheromone += Q / length` → Deposit (shorter routes get MORE pheromone).

❓ Quick Viva Questions:

Q: What happens if evaporation rate ($ ho$) is 0?
A: Pheromone accumulates indefinitely without fading, leading to premature convergence on bad paths.

TASK 6

Map Coloring using Constraint Satisfaction (CSP)

Aim: To assign the minimum possible colors (frequencies/signals) to adjacent zones in a graph such that no two neighboring zones share the same color, using a Greedy Graph Coloring algorithm.

► Python Program

```
from collections import defaultdict

class Graph:
    def __init__(self, vertices):
        self.v = vertices
        self.graph = defaultdict(list)

    def add_edge(self, u, v):
        self.graph[u].append(v)
        self.graph[v].append(u)

    def greedy_coloring(self, max_colors=None):
        result = [-1] * self.v       # -1 indicates uncolored
        available = [True] * self.v  # available[c] = True means color c is free

        for u in range(self.v):
            # 1. Mark neighbor colors as unavailable
            for neighbor in self.graph[u]:
                if result[neighbor] != -1:
                    available[result[neighbor]] = False

            # 2. Pick the lowest available color
            color = 0
            while color < self.v:
                if available[color]: break
                color += 1
            result[u] = color

            # 3. Reset available array for next vertex
            for neighbor in self.graph[u]:
                if result[neighbor] != -1:
                    available[result[neighbor]] = True

        max_used = max(result) + 1
        if max_colors is not None:
            if max_used <= max_colors:
                print(f" With {max_colors} colors: Coloring is possible!")
            else:
                print(f" With {max_colors} colors: Coloring is NOT possible!")
        return result
```

💡 **Easy Memory Hook & How to Remember the Code:**

- **Remember the 3-Step Dance:**

1. Look at neighbors: Mark their colors as `False` in `available`.
2. Find lowest color index that is still `True` → Assign to current node.
3. Reset neighbor colors back to `True` for the next node!

- **Variables to Remember:** `result = [-1]*V` (stores assigned color), `available = [True]*V` (scratchpad).

❓ Quick Viva Questions:

Q: What is the Chromatic Number of a graph?
A: The minimum number of colors needed to color the graph such that no two adjacent vertices have the same color.

TASK 7

Monkey Banana Problem in Goal Stack Planning

Aim: To simulate a robot performing goal-oriented actions (moving to location, climbing a ladder, and retrieving a tool from a shelf) using state transitions and goal stack verification in Python.

► Python Program

```
# Initial states
robot_location = "A"
ladder_location = "B"
tool_location = "C"
robot_has_tool = False
robot_on_ladder = False

print("Robot starting at", robot_location)

# Step 1: Move to ladder
robot_location = ladder_location
print("Robot picked up the ladder")
print("Robot moved to tool location", robot_location)
robot_on_ladder = True

# Step 2: Robot climbs and picks tool
print("Robot placed ladder and climbed up")
robot_has_tool = True
print("Robot picked tool from shelf")
robot_on_ladder = False

# Step 3: Climb down and goal check
print("Robot climbed down safely")
print("In Goal state Reached:")
print("Robot has tool:", robot_has_tool)
```

💡 Easy Memory Hook & How to Remember the Code:

- **5 State Variables:** 3 positions (`A, B, C`) and 2 booleans (`has_tool=False`, `on_ladder=False`).
- **3 Action Sequences (STRIPS operators):**
    1. `MOVE(A → B)` and drag ladder to tool.
    2. `CLIMB(ladder)` and `GRASP(tool)` → `has_tool = True`.
    3. `DESCEND(ladder)` → `on_ladder = False` → Check goal!

❓ Quick Viva Questions:

Q: What is Goal Stack Planning?

A: A classical AI planning algorithm that uses a stack of subgoals and STRIPS operators (Preconditions, Add-list, Delete-list) to reach a desired target state.

TASK 8

N-Queens Problem using Backtracking

> **Aim:** To place N non-attacking queens on an $N imes N$ chessboard such that no two queens share the same row, column, or diagonal, using recursive backtracking.

► Python Program

```
def solve_n_queens(n=4):
    solutions = []
    cols = set()       # Track attacked columns
    diag1 = set()      # Track positive diagonals: (r + c)
    diag2 = set()      # Track negative diagonals: (r - c)
    board = [-1] * n

    def backtrack(r=0):
        if r == n:
            solutions.append(board.copy())
            return
        for c in range(n):
            if c in cols or (r + c) in diag1 or (r - c) in diag2:
                continue   # Conflict detected!

            # Place Queen
            cols.add(c); diag1.add(r + c); diag2.add(r - c)
            board[r] = c

            backtrack(r + 1)  # Recurse to next row

            # Backtrack: Remove Queen
            cols.remove(c); diag1.remove(r + c); diag2.remove(r - c)
            board[r] = -1

    backtrack(0)
    return solutions

def format_solutions(sol):
    n = len(sol[0])
    res = []
    for s in sol:
        grid = ['.' * n for _ in range(n)]
        for r in range(n):
            row = list(grid[r]); row[s[r]] = 'Q'; grid[r] = ''.join(row)
        res.append('\n'.join(grid))
    return '\n\n'.join(res)
```

> 💡 **Easy Memory Hook & How to Remember the Code:**
>
> - **The 3 Sets Trick (No nested safety check loops needed!):**
>
> 1. `cols.add(c)` → Checks column conflict.
>
> 2. `diag1.add(r + c)` → Checks $/$-diagonal (sum $r+c$ is constant).

3. `diag2.add(r - c)` → Checks $\backslash$-diagonal (difference $r-c$ is constant).

- **Always clean up:** Add before recursing, `.remove()` immediately after recursion returns!

❓ Quick Viva Questions:

Q: How many solutions exist for 4-Queens?
A: Exactly 2 distinct solutions (reflection of each other). For 8-Queens, there are 92 solutions.

TASK 9

Intelligent Chatbot System using Conversational AI

Aim: To build an intelligent chatbot system using Python and OpenAI/NLP principles to demonstrate semantic intent analysis, context preservation, and conversational response generation.

► Python Program

```
# Simple ChatGPT & Assistant System
def run_simple_chatgpt():
    print("--- 9a: Simple ChatGPT using OpenAI ---")
    query = "Give me 3 ideas that i could build using openai apis"
    print(f"User Query: {query}\n")

    ideas = (
        "1. Personalized Content Recommendation System: AI suggestions based on user history.\n"
        "2. Intelligent Chatbot: Automated customer queries and transaction completions.\n"
        "3. Fraud Detection System: Identifies fraudulent transactions using anomaly classification."
    )
    print("Output:\n", ideas)

def run_chatgpt_assistant():
    print("\n--- 9b: ChatGPT Assistant using OpenAI ---")
    system_msg = "Nila's personal chatbot"
    print(f"What type of chatbot would you like to create?\n{system_msg}")
    print("Your new assistant is ready! Type your query")
    print("User: Hello! How can you assist me today?")
    print("Assistant: Hello Nila! I am your personal AI assistant. I can help organize tasks.")

if __name__ == "__main__":
    run_simple_chatgpt()
    run_chatgpt_assistant()
```

💡 Easy Memory Hook & How to Remember the Code:

- **Chat Roles Architecture:**
  - `system`: Defines persona and personality of the bot.
  - `user`: User query message.
  - `assistant`: Model-generated reply appended to conversation history.
- **3 Applications to mention:** Recommendation Engine, Customer Support Bot, Fraud Anomaly Detector.

❓ Quick Viva Questions:

Q: What is the purpose of Dialogflow / NLP in chatbots?
A: It parses unstructured user text into structured **Intents** (what user wants) and **Entities** (parameters/data).

TASK 10

Simple Fact Verification Engine in Python

Aim: To implement a knowledge-base fact verification system in Python that parses statement strings and validates their truth value against known conditional rules.

► Python Program

```
# Define a list of facts
facts = [
    "john_is_cold.",              # john is cold
    "raining.",                   # it is raining
    "john_Forgot_His_Raincoat.",  # john forgot his raincoat
    "fred_lost_his_car_keys.",    # fred lost his car keys
    "peter_footballer."           # peter plays football
]

def verify_fact(fact):
    fact = fact.rstrip(".")       # Strip trailing punctuation

    # Knowledge Base Rules
    if fact == "john_Forgot_His_Raincoat":
        return True
    elif fact == "raining":
        return True
    elif fact == "foggy":
        return True
    elif fact == "Cloudy":
        return False
    else:
        return False

if __name__ == "__main__":
    for fact in facts:
        if verify_fact(fact):
            print(f"{fact} - Yes")
        else:
            print(f"{fact} - No")
```

💡 Easy Memory Hook & How to Remember the Code:

- **Crucial Step:** Always strip the trailing dot with `fact.rstrip(".")` before matching strings!
- **Rules:** Only `"john_Forgot_His_Raincoat"` and `"raining"` return `True` (→ `Yes`). All other facts return `False` (→ `No`).

❓ Quick Viva Questions:

Q: What AI concept does this represent?
A: **Rule-based Expert Systems** and propositional logic knowledge representation.